Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «Криптография и сетевая безопасность»
Тема: «Исследование алгоритмов генерации псевдослучайных последовательностей и методов тестирования чисел на простоту»

Выполнил:
Студент 4 курса
Группы АС-66
Гончерёнок К.А.
Проверила:
Хацкевич А.С.

Брест 2026

Цель работы: освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.

Вариант 2

Общая схема выполнения (Индивидуальный вариант): задание выполняется строго в соответствии со своим вариантом. Все задачи объединены в единый программный комплекс (консольное приложение).

Этап 1. Генерация и тестирование псевдослучайных последовательностей. В рамках данного этапа необходимо разработать программный модуль генерации последовательностей и провести их первичный анализ. Требования:

- реализовать алгоритм генерации по варианту в виде программной функции/класса, способной выдавать последовательность произвольной длины;
- подобрать параметры генератора (a,c,N,x0 и др.) так, чтобы добиться максимального периода последовательности Tmax (согласно теоретическим условиям из методички);
- провести базовое статистическое тестирование полученной последовательности (например, оценить частотность появления элементов, равномерность распределения).

Генератор: Комбинирование с помощью псевдослучайного прореживания.

Этап 2. Тестирование на простоту и генерация больших простых чисел. На втором этапе сгенерированные на первом этапе псевдослучайные числа (или базовые механизмы генерации) интегрируются в модуль поиска и проверки чисел на простоту. Требования:

- реализовать стандартную схему генерации кандидата в простые числа:
  - сгенерировать случайное p-битовое число n (используя генератор из Этапа 1);
  - установить старший и младший биты в 1 (для гарантии разрядности и нечетности);
  - выполнить предварительную фильтрацию (просеивание) делением на малые простые числа (2, 3, 5, 7, ..., вплоть до 256 или 2000), что отсекает до 80% составных чисел без тяжелых тестов;
- интегрировать целевой тест из таблицы вариантов (вместо стандартного Рабина-Миллера или в дополнение к нему) для финальной проверки кандидата. Тест Леманна.

main.py:

```
import argparse
import random
import time

from src.lfsr import is_primitive, poly_degree
from src.sg_generator import SGGenerator
from src.stats import monobit_test, block_test, chi2_critical, runs_test


POLYNOMIALS = {
    1:  0b10001111,
    2:  0b10000011,
    3:  0b10001001,
    11: 0b100000000101,
}


def _small_primes(limit: int = 2000) -> list[int]:
    sieve = bytearray([1]) * (limit + 1)
    sieve[0:2] = b"\x00\x00"
    i = 2
    while i * i <= limit:
        if sieve[i]:
            sieve[i * i::i] = bytearray(len(sieve[i * i::i]))
        i += 1
    return [i for i in range(limit + 1) if sieve[i]]


SMALL_PRIMES = _small_primes(2000)


def lehmann_test(p: int, t: int = 10) -> bool:
    if p < 5 or p % 2 == 0:
        return p in (2, 3)
    for _ in range(t):
        a = random.randint(2, p - 2)
        r = pow(a, (p - 1) // 2, p)
        if r != 1 and r != p - 1:
            return False
    return True


def generate_prime(bits: int, sg, t: int = 10,
                   max_attempts: int = 1_000_000) -> tuple[int, dict]:
    stats = {"candidates": 0, "sieve_rejected": 0, "lehmann_rejected": 0}
    for _ in range(max_attempts):
        stats["candidates"] += 1
        bits_list = sg.next_bits(bits)
        n = 0
        for b in bits_list:
            n = (n << 1) | b
        n |= (1 << (bits - 1))
        n |= 1
        rejected = False
        for q in SMALL_PRIMES:
            if q >= n:
                break
            if n % q == 0:
                stats["sieve_rejected"] += 1
                rejected = True
                break
        if rejected:
            continue
        if lehmann_test(n, t):
            return n, stats
```

```
            stats["lehmann_rejected"] += 1
        raise RuntimeError("не удалось сгенерировать простое число")


def check_primitive() -> None:
    print("=" * 60)
    print("Проверка примитивности многочленов")
    print("=" * 60)
    for idx, poly in POLYNOMIALS.items():
        n = poly_degree(poly)
        prim = is_primitive(poly)
        print(f"f{idx}(x), степень {n}, маска 0b{poly:b} — "
              f"{'ПРИМИТИВЕН' if prim else 'не примитивен'}")
    print()


def run_sg(g1_idx: int, g2_idx: int, seed1: int, seed2: int,
           bits_count: int) -> dict:
    poly1 = POLYNOMIALS[g1_idx]
    poly2 = POLYNOMIALS[g2_idx]
    gen = SGGenerator(poly1, seed1, poly2, seed2)

    print(f"G1: f{g1_idx} = 0b{poly1:b}, seed = {seed1}, "
          f"период LFSR = {gen._g1.period}")
    print(f"G2: f{g2_idx} = 0b{poly2:b}, seed = {seed2}, "
          f"период LFSR = {gen._g2.period}")
    try:
        print(f"Теоретический период SG: {gen.theoretical_period}")
    except ValueError as e:
        print(f"⚠ {e}")

    print(f"\nГенерация {bits_count} бит...")
    t0 = time.perf_counter()
    bits = gen.next_bits(bits_count)
    dt = time.perf_counter() - t0
    print(f"Сгенерировано за {dt:.4f} с ({bits_count / dt / 1000:.1f} кбит/с)")

    print("\n" + "=" * 60)
    print("Статистические тесты")
    print("=" * 60)

    p, z = monobit_test(bits)
    print(f"Частотный: доля единиц = {p:.6f}, z = {z:.4f}")

    r = runs_test(bits)
    print(f"Серийный: runs = {r['runs']}, expected = {r['expected']:.1f}, "
          f"z = {r['z']:.4f}")

    for k in (4, 8):
        try:
            chi2 = block_test(bits, k)
            df = (1 << k) - 1
            crit = chi2_critical(df)
            verdict = "ОК" if chi2 < crit else "ОТКЛОНЕНО"
            print(f"Блочный k={k}: χ² = {chi2:.2f}, "
                  f"крит. = {crit:.2f} (df={df}) → {verdict}")
        except ValueError as e:
            print(f"Блочный k={k}: {e}")

    return {"p": p, "z": z, "runs": r["runs"], "bits": bits_count}


def main():
    ap = argparse.ArgumentParser(description="ЛР №1, вариант 2 — SG-генератор")
    ap.add_argument("--check-primitive", action="store_true")
    ap.add_argument("--g1", type=int, default=2, choices=[1, 2, 3, 11])
```

```
    ap.add_argument("--g2", type=int, default=11, choices=[1, 2, 3, 11])
    ap.add_argument("--seed1", type=int, default=1)
    ap.add_argument("--seed2", type=int, default=1)
    ap.add_argument("--bits", type=int, default=100_000)
    ap.add_argument("--prime", action="store_true")
    ap.add_argument("--prime-bits", type=int, default=32)
    ap.add_argument("--prime-t", type=int, default=10)

    args = ap.parse_args()

    if args.check_primitive:
        check_primitive()
        return

    if args.prime:
        gen = SGGenerator(POLYNOMIALS[2], args.seed1,
                          POLYNOMIALS[11], args.seed2)
        print(f"Генерация {args.prime_bits}-битного простого "
              f"(тест Лемана, t={args.prime_t})...")
        t0 = time.perf_counter()
        prime, stats = generate_prime(args.prime_bits, gen, args.prime_t)
        dt = time.perf_counter() - t0
        print(f"Найдено простое: {prime}")
        print(f"  бит: {prime.bit_length()}")
        print(f"  время: {dt:.3f} с")
        print(f"  кандидатов: {stats['candidates']}")
        print(f"  отсеяно просеиванием: {stats['sieve_rejected']}")
        print(f"  отсеяно Леманом: {stats['lehmann_rejected']}")
        return

    check_primitive()
    run_sg(args.g1, args.g2, args.seed1, args.seed2, args.bits)


if __name__ == "__main__":
    main()
```

lsfr.py:

```
def poly_degree(poly: int) -> int:
    return poly.bit_length() - 1


def poly_mod(a: int, b: int) -> int:
    if b == 0:
        raise ZeroDivisionError("деление на нулевой многочлен")
    db = poly_degree(b)
    while a and poly_degree(a) >= db:
        a ^= b << (poly_degree(a) - db)
    return a


def poly_mulmod(a: int, b: int, mod: int) -> int:
    result = 0
    dmod = poly_degree(mod)
    while b:
        if b & 1:
            result ^= a
        b >>= 1
        a <<= 1
        if poly_degree(a) >= dmod:
            a ^= mod << (poly_degree(a) - dmod)
    return poly_mod(result, mod)
```

```
def poly_powmod(base: int, exp: int, mod: int) -> int:
    result = 1
    base = poly_mod(base, mod)
    while exp:
        if exp & 1:
            result = poly_mulmod(result, base, mod)
        base = poly_mulmod(base, base, mod)
        exp >>= 1
    return result


def _poly_gcd(a: int, b: int) -> int:
    while b:
        a, b = b, poly_mod(a, b)
    return a


def _prime_factors(n: int) -> list[int]:
    factors = []
    d = 2
    while d * d <= n:
        if n % d == 0:
            factors.append(d)
            while n % d == 0:
                n //= d
        d += 1
    if n > 1:
        factors.append(n)
    return factors


def is_irreducible(poly: int) -> bool:
    n = poly_degree(poly)
    if n < 1:
        return False
    if n == 1:
        return True

    x = 0b10

    if poly_powmod(x, 1 << n, poly) != x:
        return False

    for d in range(1, n):
        if n % d != 0:
            continue
        h = poly_powmod(x, 1 << d, poly) ^ x
        if h == 0:
            return False
        if _poly_gcd(h, poly) != 1:
            return False
    return True


def is_primitive(poly: int) -> bool:
    n = poly_degree(poly)
    if not is_irreducible(poly):
        return False
    if n == 1:
        return True
    order = (1 << n) - 1
    x = 0b10  # ← и здесь тоже
    for p in _prime_factors(order):
        if poly_powmod(x, order // p, poly) == 1:
            return False
    return True
```

```
class LFSR:

    __slots__ = ("_state", "_mask", "_tap_mask", "_n")

    def __init__(self, poly: int, seed: int):
        if not is_primitive(poly):
            raise ValueError(f"Многочлен 0b{poly:b} не примитивен")
        n = poly_degree(poly)
        self._n = n
        self._mask = (1 << n) - 1
        self._tap_mask = poly & self._mask
        if seed == 0 or seed > self._mask:
            raise ValueError("seed должен быть в [1, 2^n – 1]")
        self._state = seed & self._mask

    def next_bit(self) -> int:
        out = self._state & 1
        feedback = (self._state & self._tap_mask).bit_count() & 1
        self._state = (self._state >> 1) | (feedback << (self._n - 1))
        return out

    def next_bits(self, count: int) -> list[int]:
        return [self.next_bit() for _ in range(count)]

    @property
    def period(self) -> int:
        return (1 << self._n) – 1
```

sg_generator.py:

```
from .lfsr import LFSR, is_primitive

class SGGenerator:

    __slots__ = ("_g1", "_g2", "_n", "_m")

    def __init__(self, poly1: int, seed1: int, poly2: int, seed2: int):
        if not is_primitive(poly1):
            raise ValueError(f"G1: многочлен 0b{poly1:b} не примитивен")
        if not is_primitive(poly2):
            raise ValueError(f"G2: многочлен 0b{poly2:b} не примитивен")
        self._g1 = LFSR(poly1, seed1)
        self._g2 = LFSR(poly2, seed2)
        self._n = self._g1.period   # 2^n – 1
        self._m = self._g2.period   # 2^m – 1

    def next_bit(self) -> int:
        while True:
            a = self._g1.next_bit()
            s = self._g2.next_bit()
            mask = -s
            if mask & 1:
                return a

    def next_bits(self, count: int) -> list[int]:
        out = []
        append = out.append
        g1_next = self._g1.next_bit
        g2_next = self._g2.next_bit
        while len(out) < count:
            a = g1_next()
            s = g2_next()
            if s:
```

```
                append(a)
        return out

    @property
    def theoretical_period(self) -> int:
        from math import gcd
        if gcd(self._n, self._m) != 1:
            raise ValueError(
                f"Периоды {self._n} и {self._m} не взаимно просты — "
                f"теорема о периоде SG не применима"
            )
        return self._n * (1 << (self._m.bit_length() - 1))
```

stats.py:

```
import math


def monobit_test(bits: list[int]) -> tuple[float, float]:
    n = len(bits)
    ones = sum(bits)
    p = ones / n
    # z = (2·ones − n) / sqrt(n) ~ N(0,1) при H0
    z = (2 * ones - n) / math.sqrt(n)
    return p, z


def block_test(bits: list[int], block_size: int = 8) -> float:
    n = len(bits)
    n_blocks = n // block_size
    if n_blocks < 5 * (1 << block_size):
        raise ValueError("Слишком мало данных для блочного теста")

    counts = [0] * (1 << block_size)
    for i in range(n_blocks):
        val = 0
        base = i * block_size
        for j in range(block_size):
            val = (val << 1) | bits[base + j]
        counts[val] += 1

    expected = n_blocks / (1 << block_size)
    chi2 = sum((c - expected) ** 2 / expected for c in counts)
    return chi2


def chi2_critical(df: int, alpha: float = 0.05) -> float:
    z = {0.05: 1.6449, 0.01: 2.3263, 0.10: 1.2816}[alpha]
    t = 2.0 / (9.0 * df)
    return df * (1 - t + z * math.sqrt(t)) ** 3


def runs_test(bits: list[int]) -> dict:
    n = len(bits)
    runs = 1
    for i in range(1, n):
        if bits[i] != bits[i - 1]:
            runs += 1
    ones = sum(bits)
    zeros = n - ones
    expected = 1 + 2 * ones * zeros / n
    var = (2 * ones * zeros * (2 * ones * zeros - n)) / (n * n * (n - 1))
    z = (runs - expected) / math.sqrt(var) if var > 0 else 0.0
    return {"runs": runs, "expected": expected, "z": z}
```

Вывод: освоили программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.